МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ АВТОМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ
Кафедра автоматизированных систем управления

ОТЧЕТ

по дисциплине Современные технологии разработки программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА № 9

Тема: Работа со стандартными приложениями

Выполнил:
Епифанцев Г.А.
Группа АПМ-25

Подпись: _____ Дата: ___

Проверил:
Эстрайх И.В.

Оценка _____

Подпись: _____ Дата: ___

Новосибирск 2026

СОДЕРЖАНИЕ

Постановка задачи **3**

1 Ход работы **4**

1.1 Главное меню приложения 4

1.2 Проверка подключения к Интернету 5

1.3 Передача номера в приложение «Телефон» 6

1.4 Подготовка электронного письма 7

1.5 Выбор веб-ссылки и браузера 9

1.6 Встроенный браузер WebView 10

Заключение **11**

Приложение А. Исходный код **12**

ПОСТАНОВКА ЗАДАЧИ

Цель работы

Научиться вызывать стандартные приложения Android, передавать им данные через Intent и создавать встроенный браузер на основе WebView.

Задание

1. Проверить наличие подключения к Интернету. Для каждого состояния показать свой значок и всплывающее сообщение.
2. Предложить несколько телефонных номеров и передать выбранный номер в системное приложение «Телефон».
3. Создать форму с адресом получателя, темой и текстом письма. Добавить очистку формы и подготовку письма в выбранном почтовом клиенте.
4. Предложить несколько веб-ссылок и выбор браузера. Реализовать встроенный WebView, переходы по ссылкам внутри него и возврат к предыдущей странице.
5. Включить в отчёт цель работы, код программ, скриншоты результатов и вывод.

1 Ход работы

1.1 Главное меню приложения

Четыре функции объединены в проекте Lab9StandardApps на Kotlin. Главная активность открывает отдельные экраны для проверки Интернета, выбора номера, подготовки письма и выбора веб-ссылки. Для перехода на нужный экран создаётся явное намерение с классом соответствующей активности.

```
findViewById<android.view.View>(R.id.networkButton).setOnClickListener {
    startActivity(Intent(this, NetworkActivity::class.java))
}
```

Главное меню с четырьмя разделами показано на рисунке 1.

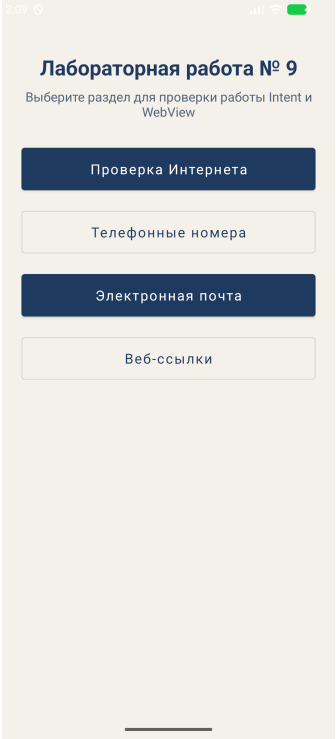


Рисунок 1 — Главное меню приложения

1.2 Проверка подключения к Интернету

Состояние активной сети определяется через ConnectivityManager. Проверяются одновременно возможности NET_CAPABILITY_INTERNET и NET_CAPABILITY_VALIDATED. Первая означает, что сеть рассчитана на доступ в Интернет, вторая подтверждает успешную системную проверку связи.

```
val network = manager.activeNetwork ?: return false
val capabilities = manager.getNetworkCapabilities(network) ?: return false
return capabilities.hasCapability(NetworkCapabilities.
    NET_CAPABILITY_INTERNET) &&
     capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED
    )
```

При нажатии на кнопку приложение обновляет значок Wi-Fi и показывает Toast. Подключение обозначено бирюзовым значком на рисунке 2. После отключения сети значок меняет цвет, а текст сообщает об отсутствии соединения (рисунок 3).

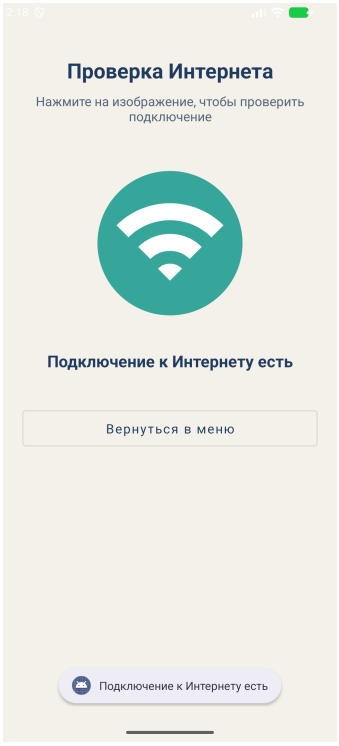


Рисунок 2 — Подключение к Интернету есть

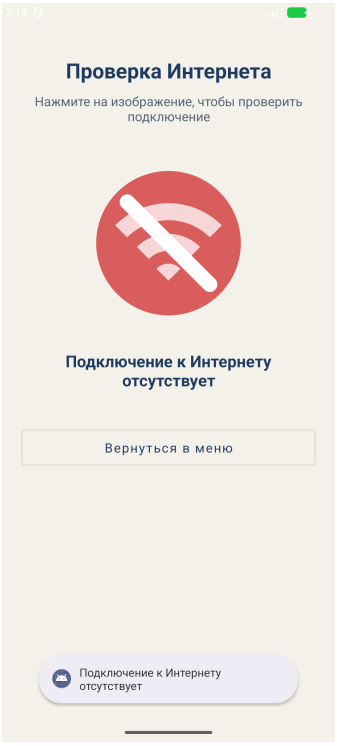


Рисунок 3 — Подключение отсутствует

1.3 Передача номера в приложение «Телефон»

Пользователь выбирает один из трёх номеров в группе RadioGroup. Обработчик получает номер из строкового ресурса и передаёт его системному номеронабирателю через ACTION_DIAL с URI схемы tel. Пользователь может проверить цифры перед вызовом.

```
val dialIntent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:$selectedNumber"
    ))
try {
    startActivity(dialIntent)
} catch (_: ActivityNotFoundException) {
    Toast.makeText(this, R.string.dialer_not_found, Toast.LENGTH_LONG).show
    ()
}
```

На рисунке 4 выбран номер деканата. На рисунке 5 тот же номер заполнен в системном приложении.

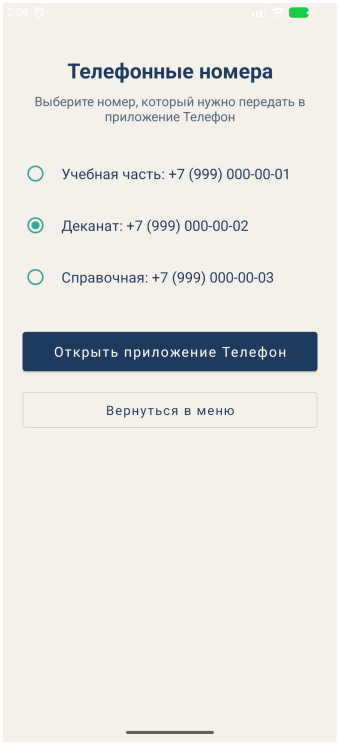


Рисунок 4 — Выбор телефонного номера

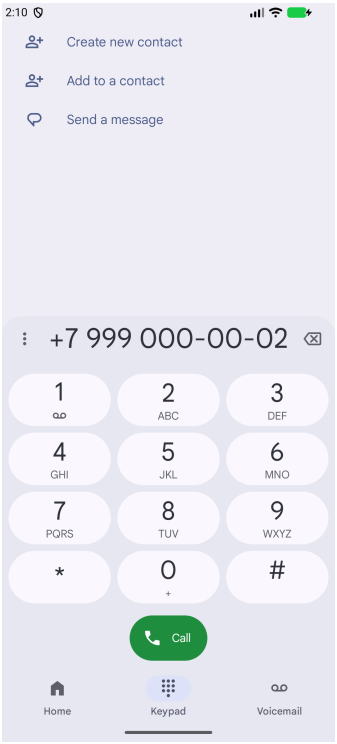


Рисунок 5 — Номер в системном номеронаборателе

1.4 Подготовка электронного письма

Форма содержит поля получателя, темы и сообщения. Кнопка «Очистить» стирает содержимое всех трёх полей. При нажатии «Отправить» приложение проверяет заполнение и открывает выбор почтового клиента.

В поле адреса вводится получатель, в остальных полях тема и текст сообщения. На рисунке 6 показана пустая форма, на рисунке 7 введены данные учебного письма.

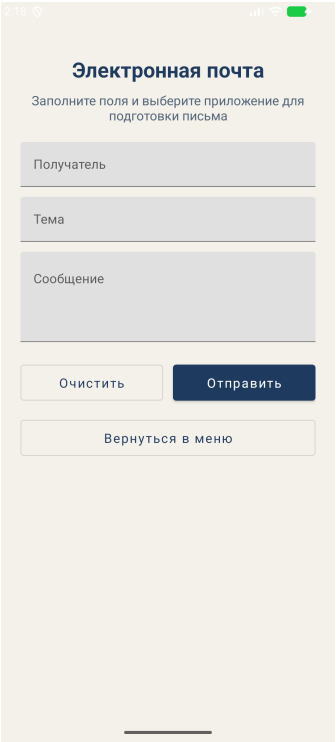


Рисунок 6 — Пустая форма письма

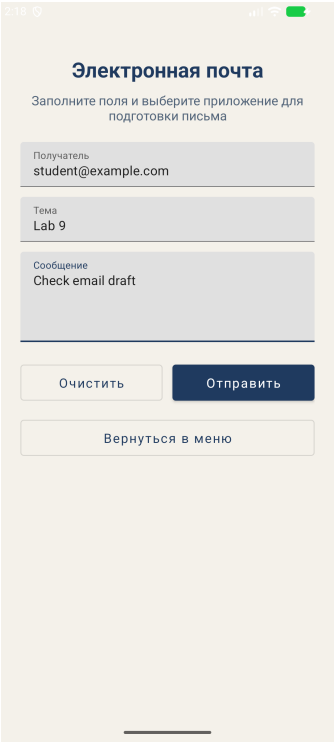


Рисунок 7 — Заполненная форма письма

Для подготовки письма создаётся намерение ACTION_SENDTO со схемой mailto. Получатель, тема и текст передаются в URI и дополнительных полях намерения. Система показывает выбор приложений, которые обрабатывают почтовые ссылки. В случае отсутствия клиента приложение выводит сообщение.

```
val mailIntent = Intent(Intent.ACTION_SENDTO).apply {
    data = Uri.parse(
        "mailto:${Uri.encode(recipient, "@")}?subject=${Uri.encode(subject)
    }" +
            "&body=${Uri.encode(message)}",
    )
    putExtra(Intent.EXTRA_EMAIL, arrayOf(recipient))
    putExtra(Intent.EXTRA_SUBJECT, subject)
    putExtra(Intent.EXTRA_TEXT, message)
}
```

На рисунке 8 Android предлагает Thunderbird и Gmail. После выбора Thunderbird открывается редактор с заполненными полями получателя, темы и текста (рисунок 9).

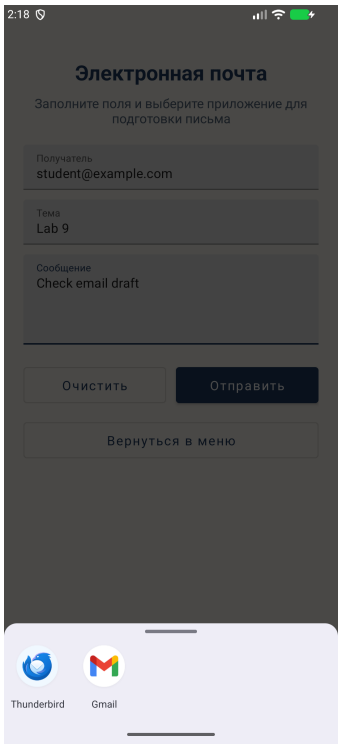


Рисунок 8 — Выбор почтового клиента

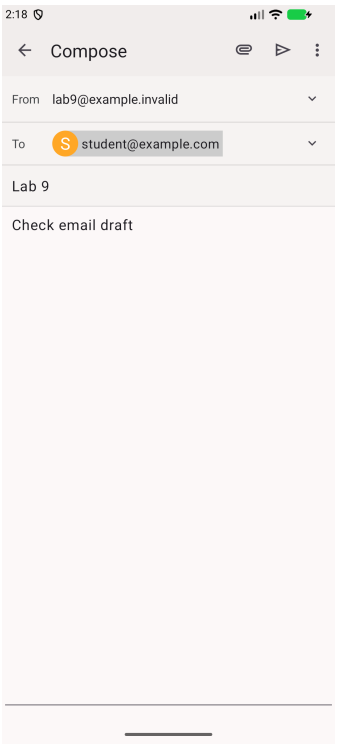


Рисунок 9 — Заполненный черновик в Thunderbird

1.5 Выбор веб-ссылки и браузера

На экране доступны три HTTPS-ссылки. После выбора создаётся намерение ACTION_VIEW с адресом страницы. В системное окно выбора добавляется встроенная активность BrowserActivity. Внешний браузер передаётся Android для поиска обработчиков того же адреса.

```
val viewIntent = Intent(Intent.ACTION_VIEW, uri)
val internalBrowserIntent = Intent(this, BrowserActivity::class.java).apply
    {
    action = Intent.ACTION_VIEW
    data = uri
}
val chooserIntent = Intent.createChooser(viewIntent, getString(R.string.
   choose_browser)).apply {
    putExtra(Intent.EXTRA_INITIAL_INTENTS, arrayOf(internalBrowserIntent))
}
```

Список страниц показан на рисунке 10. После нажатия «Открыть ссылку» Android предлагает выбрать обработчик (рисунок 11).

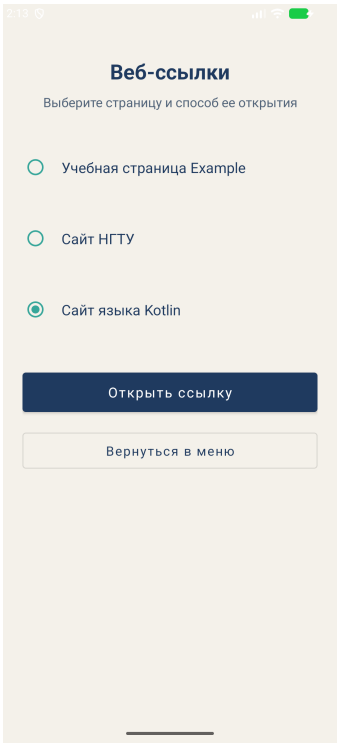


Рисунок 10 — Выбор веб-страницы

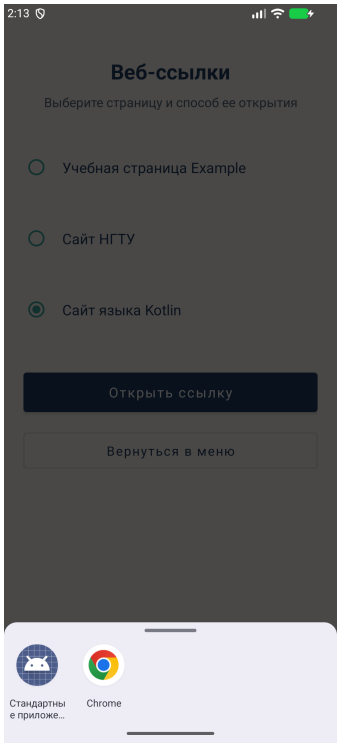


Рисунок 11 — Системный выбор браузера

1.6 Встроенный браузер WebView

Активность получает адрес из намерения и загружает его в WebView. WebViewClient оставляет навигацию по ссылкам внутри экрана. Адресная строка обновляется при переходе, а кнопки «Назад» и «Вперёд» используют историю WebView. Системная кнопка возврата сначала открывает предыдущую веб-страницу, а на первой странице возвращает к списку ссылок.

```
override fun handleOnBackPressed() {
    if (webView.canGoBack()) {
        webView.goBack()
    } else {
        isEnabled = false
        onBackPressedDispatcher.onBackPressed()
    }
}
```

На рисунке 12 открыта главная страница сайта Kotlin. Нажатие Get started загружает страницу документации в том же окне (рисунок 13). После перехода становится доступна кнопка «Назад».

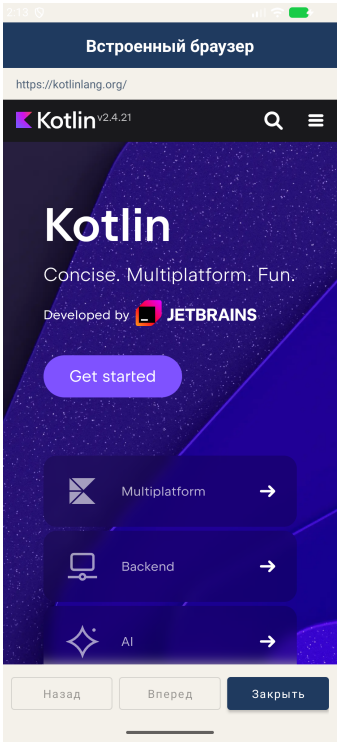


Рисунок 12 — Исходная страница в WebView

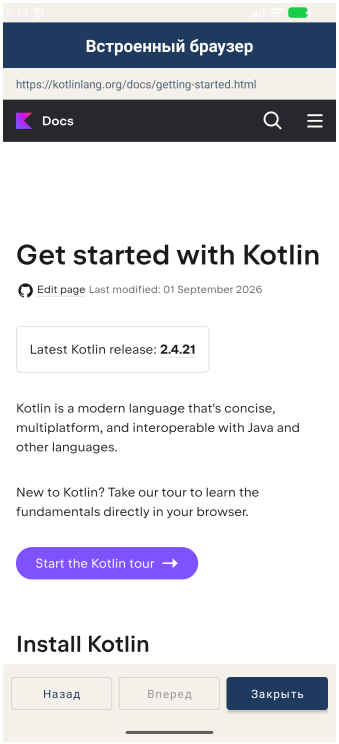


Рисунок 13 — Переход по ссылке внутри WebView

ЗАКЛЮЧЕНИЕ

В приложении реализованы проверка подключения к Интернету, передача номера в приложение «Телефон» и подготовка письма в выбранном почтовом клиенте. Данные передаются через Intent. Для веб-ссылок доступен системный выбор браузера, а встроенный WebView открывает страницы и позволяет возвращаться по истории.

ПРИЛОЖЕНИЕ А

Исходный код

MainActivity.kt

```
package ru.nstu.lab9.standardapps

import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        findViewById<android.view.View>(R.id.networkButton).
   setOnClickListener {
            startActivity(Intent(this, NetworkActivity::class.java))
        }
        findViewById<android.view.View>(R.id.phoneButton).
   setOnClickListener {
            startActivity(Intent(this, PhoneActivity::class.java))
        }
        findViewById<android.view.View>(R.id.emailButton).
   setOnClickListener {
            startActivity(Intent(this, EmailActivity::class.java))
        }
        findViewById<android.view.View>(R.id.linksButton).
   setOnClickListener {
            startActivity(Intent(this, LinksActivity::class.java))
        }
    }
}
```

NetworkActivity.kt

```
package ru.nstu.lab9.standardapps

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities
import android.os.Bundle
import android.widget.ImageButton
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class NetworkActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_network)

        val networkImage = findViewById<ImageButton>(R.id.networkImage)
        networkImage.setOnClickListener {
            showNetworkState(networkImage, true)
        }
        findViewById<android.view.View>(R.id.backButton).setOnClickListener
    {
            finish()
        }

        showNetworkState(networkImage, false)
    }

    private fun showNetworkState(networkImage: ImageButton, showToast:
   Boolean) {
        val connected = isInternetAvailable()
        val message = if (connected) {
            networkImage.setImageResource(R.drawable.ic_wifi_online)
            getString(R.string.network_online)
        } else {
            networkImage.setImageResource(R.drawable.ic_wifi_offline)
            getString(R.string.network_offline)
        }

        findViewById<TextView>(R.id.networkStatus).text = message
        if (showToast) {
            Toast.makeText(this, message, Toast.LENGTH_LONG).show()
        }
    }

    private fun isInternetAvailable(): Boolean {
        val manager = getSystemService(Context.CONNECTIVITY_SERVICE) as
   ConnectivityManager
        val network = manager.activeNetwork ?: return false
        val capabilities = manager.getNetworkCapabilities(network) ?:
   return false
        return capabilities.hasCapability(NetworkCapabilities.
   NET_CAPABILITY_INTERNET) &&
            capabilities.hasCapability(NetworkCapabilities.
   NET_CAPABILITY_VALIDATED)
    }
}
```

PhoneActivity.kt

```
package ru.nstu.lab9.standardapps

import android.content.ActivityNotFoundException
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.RadioGroup
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class PhoneActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_phone)

        findViewById<android.view.View>(R.id.openDialerButton).
    setOnClickListener {
            openSelectedNumber()
        }
        findViewById<android.view.View>(R.id.phoneBackButton).
    setOnClickListener {
            finish()
        }
    }

    private fun openSelectedNumber() {
        val selectedNumber = when (findViewById<RadioGroup>(R.id.
    phoneNumbers).checkedRadioButtonId) {
            R.id.firstNumber -> getString(R.string.first_phone_value)
            R.id.secondNumber -> getString(R.string.second_phone_value)
            R.id.thirdNumber -> getString(R.string.third_phone_value)
            else -> null
        }

        if (selectedNumber == null) {
            Toast.makeText(this, R.string.choose_phone_number, Toast.
    LENGTH_SHORT).show()
            return
        }

        val dialIntent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:
    $selectedNumber"))
        try {
            startActivity(dialIntent)
        } catch (_: ActivityNotFoundException) {
            Toast.makeText(this, R.string.dialer_not_found, Toast.
    LENGTH_LONG).show()
        }
    }
}
```

EmailActivity.kt

```
package ru.nstu.lab9.standardapps

import android.content.ActivityNotFoundException
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.textfield.TextInputEditText

class EmailActivity : AppCompatActivity() {
    private lateinit var recipientInput: TextInputEditText
    private lateinit var subjectInput: TextInputEditText
    private lateinit var messageInput: TextInputEditText

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_email)

        recipientInput = findViewById(R.id.recipientInput)
        subjectInput = findViewById(R.id.subjectInput)
        messageInput = findViewById(R.id.messageInput)

        findViewById<android.view.View>(R.id.clearEmailButton).
   setOnClickListener {
            clearForm()
        }
        findViewById<android.view.View>(R.id.sendEmailButton).
   setOnClickListener {
            openEmailApp()
        }
        findViewById<android.view.View>(R.id.emailBackButton).
   setOnClickListener {
            finish()
        }
    }

    private fun clearForm() {
        recipientInput.text?.clear()
        subjectInput.text?.clear()
        messageInput.text?.clear()
        recipientInput.requestFocus()
        Toast.makeText(this, R.string.form_cleared, Toast.LENGTH_SHORT).
   show()
    }

    private fun openEmailApp() {
        val recipient = recipientInput.text.toString().trim()
        val subject = subjectInput.text.toString().trim()
        val message = messageInput.text.toString().trim()

        if (recipient.isBlank() || subject.isBlank() || message.isBlank())
   {
            Toast.makeText(this, R.string.fill_email_fields, Toast.
   LENGTH_SHORT).show()
            return
        }
```

```
        val mailIntent = Intent(Intent.ACTION_SENDTO).apply {
            data = Uri.parse(
                "mailto:${Uri.encode(recipient, "@")}?subject=${Uri.encode(
    subject)}" +
                    "&body=${Uri.encode(message)}",
            )
            putExtra(Intent.EXTRA_EMAIL, arrayOf(recipient))
            putExtra(Intent.EXTRA_SUBJECT, subject)
            putExtra(Intent.EXTRA_TEXT, message)
        }
        if (mailIntent.resolveActivity(packageManager) == null) {
            Toast.makeText(this, R.string.email_app_not_found, Toast.
    LENGTH_LONG).show()
            return
        }

        try {
            startActivity(Intent.createChooser(mailIntent, getString(R.
    string.choose_email_app)))
        } catch (_: ActivityNotFoundException) {
            Toast.makeText(this, R.string.email_app_not_found, Toast.
    LENGTH_LONG).show()
        }
    }
}
```

LinksActivity.kt

```
package ru.nstu.lab9.standardapps

import android.content.ActivityNotFoundException
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.RadioGroup
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class LinksActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_links)

        findViewById<android.view.View>(R.id.openLinkButton).
   setOnClickListener {
            openSelectedLink()
        }
        findViewById<android.view.View>(R.id.linksBackButton).
   setOnClickListener {
            finish()
        }
    }

    private fun openSelectedLink() {
        val url = when (findViewById<RadioGroup>(R.id.webLinks).
   checkedRadioButtonId) {
            R.id.exampleLink -> getString(R.string.example_link_value)
            R.id.nstuLink -> getString(R.string.nstu_link_value)
            R.id.kotlinLink -> getString(R.string.kotlin_link_value)
            else -> null
        }

        if (url == null) {
            Toast.makeText(this, R.string.choose_web_link, Toast.
   LENGTH_SHORT).show()
            return
        }

        val uri = Uri.parse(url)
        val viewIntent = Intent(Intent.ACTION_VIEW, uri)
        val internalBrowserIntent = Intent(this, BrowserActivity::class.
   java).apply {
            action = Intent.ACTION_VIEW
            data = uri
        }
        val chooserIntent = Intent.createChooser(viewIntent, getString(R.
   string.choose_browser)).apply {
            putExtra(Intent.EXTRA_INITIAL_INTENTS, arrayOf(
   internalBrowserIntent))
        }
        try {
            startActivity(chooserIntent)
        } catch (_: ActivityNotFoundException) {
            Toast.makeText(this, R.string.browser_not_found, Toast.
   LENGTH_LONG).show()
        }
    }
}
```

BrowserActivity.kt

```
package ru.nstu.lab9.standardapps

import android.graphics.Bitmap
import android.os.Bundle
import android.view.View
import android.webkit.WebResourceRequest
import android.webkit.WebView
import android.webkit.WebViewClient
import android.widget.TextView
import android.widget.Toast
import androidx.activity.OnBackPressedCallback
import androidx.appcompat.app.AppCompatActivity

class BrowserActivity : AppCompatActivity() {
    private lateinit var webView: WebView
    private lateinit var addressText: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_browser)

        webView = findViewById(R.id.webView)
        addressText = findViewById(R.id.addressText)
        webView.settings.javaScriptEnabled = true
        webView.webViewClient = object : WebViewClient() {
            override fun shouldOverrideUrlLoading(
                view: WebView,
                request: WebResourceRequest,
            ): Boolean = false

            override fun onPageStarted(view: WebView, url: String, favicon:
    Bitmap?) {
                addressText.text = url
            }

            override fun onPageFinished(view: WebView, url: String) {
                addressText.text = url
                updateNavigationButtons()
            }
        }

        findViewById<View>(R.id.browserBackButton).setOnClickListener {
            if (webView.canGoBack()) webView.goBack()
        }
        findViewById<View>(R.id.browserForwardButton).setOnClickListener {
            if (webView.canGoForward()) webView.goForward()
        }
        findViewById<View>(R.id.browserCloseButton).setOnClickListener {
            finish()
        }

        onBackPressedDispatcher.addCallback(this, object :
   OnBackPressedCallback(true) {
            override fun handleOnBackPressed() {
                if (webView.canGoBack()) {
                    webView.goBack()
                } else {
```

```
                    isEnabled = false
                    onBackPressedDispatcher.onBackPressed()
                }
            }
        })

        val initialUrl = intent.dataString ?: getString(R.string.
    example_link_value)
        if (initialUrl.startsWith("http://") || initialUrl.startsWith("
    https://")) {
            webView.loadUrl(initialUrl)
        } else {
            Toast.makeText(this, R.string.page_load_error, Toast.
    LENGTH_LONG).show()
            finish()
        }
        updateNavigationButtons()
    }

    private fun updateNavigationButtons() {
        findViewById<View>(R.id.browserBackButton).isEnabled = webView.
    canGoBack()
        findViewById<View>(R.id.browserForwardButton).isEnabled = webView.
    canGoForward()
    }

    override fun onDestroy() {
        webView.destroy()
        super.onDestroy()
    }
}
```